Python语言与系统设计

（Python序列）

Python Language & System Design

第2章 Python序列
(Python Sequence)



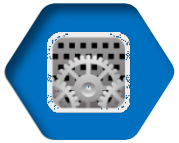
合肥工业大学 计算机与信息学院

第2章　Python序列


2.1 Python序列概述


2.2 列表


2.3 元组


2.4 字典

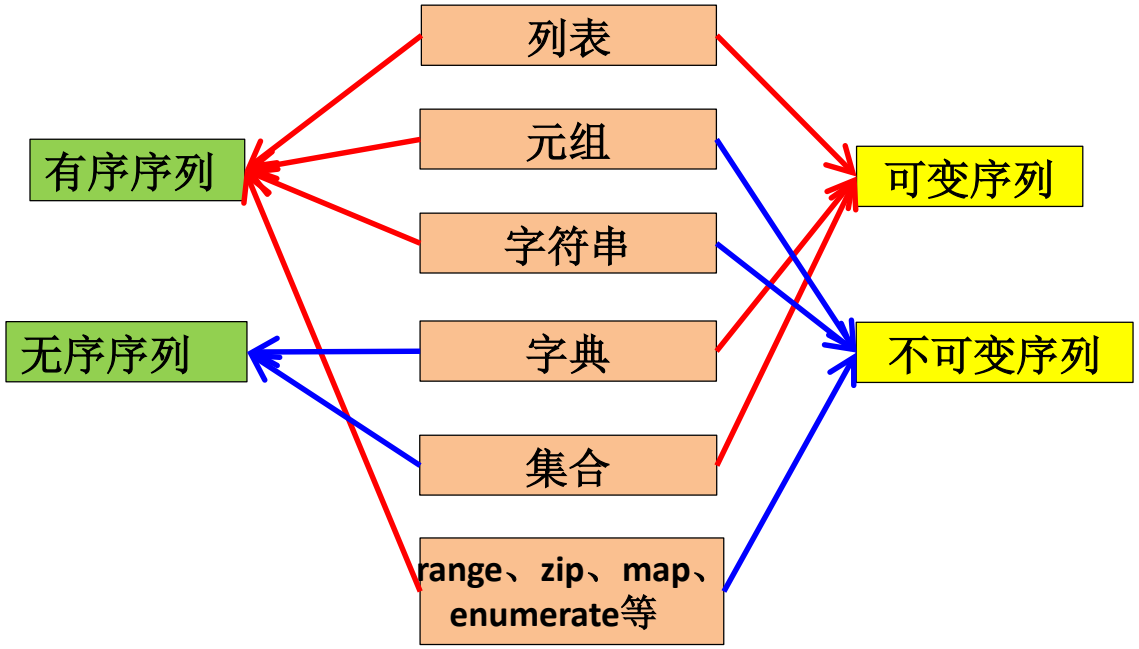

2.5 集合


2.6 复杂数据结构

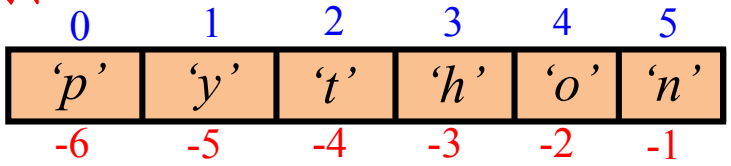

2.7 本章小结

2.1 Python序列概述

- Python序列(Python Sequence)
  - Python序列类似于其他语言中的数组，但功能要强大很多。
  - Python中常用的序列结构：



  - 列表、元组、字符串支持双向索引



注：slides参考：董付国. 《Python程序设计》(第2版). 清华大学出版社, 2018.

2.1 Python序列概述

Python序列(Python Sequence)

	列表	元组	字典	集合
类型名称	list	tuple	dict	set
定界符	方括号[]	圆括号()	大括号{}	大括号{}
是否可变	是	否	是	是
是否有序	是	是	否	否
是否支持下标	是(序号作为下标)	是(序号作为下标)	是(“键”作为下标)	否
元素分隔符	逗号	逗号	逗号	逗号
对元素形式的要求	无	无	键:值	必须可哈希
对元素值的要求	无	无	“键”必须可哈希	必须可哈希
元素是否可重复	是	是	“键”不允许重复，“值”可以重复	否
元素查找速度	非常慢	很慢	非常快	非常快
新增和删除元素速度	尾部操作快 其他位置慢	不允许	快	快

2.2 列表

➢列表(List)的基本概念

- 列表是Python中内置有序、可变序列，列表的所有元素放在一对中括号“[]”中，并使用逗号分隔开；
- 当列表元素增加或删除时，列表对象自动进行扩展或收缩内存，保证元素之间没有缝隙；
- 在Python中，一个列表中的数据类型可以各不相同
  - ✓ 例如：

```
[10, 20, 30, 40]
['frog', 'fish', 'bird']
['spam', 2.0, 5, [10, 20]]
[['file1', 200,7], ['file2', 260,9]]
```

2.2 列表

列表的常用方法

方法	说明
lst *= n	更新列表lst, 其元素重复n次
lst.append(x)	将元素x添加至列表lst尾部
lst.extend(L) 或lst += L	将列表L中所有元素添加至列表lst尾部
lst.insert(index, x)	在列表lst指定位置index处添加元素x，该位置后面的所有元素后移一个位置
lst.remove(x)	在列表lst中删除首次出现的指定元素，该元素之后的所有元素前移一个位置
lst.pop([index])	删除并返回列表lst中下标为index（默认为-1）的元素
lst.clear()	删除列表lst中所有元素，但保留列表对象
lst.index(x)	返回列表lst中第一个值为x的元素的下标，若不存在值为x的元素则抛出异常
lst.count(x)	返回指定元素x在列表lst中的出现次数
lst.reverse()	对列表lst所有元素进行逆序
lst.sort(key=None, reverse=False)	对列表lst中的元素进行排序，key用来指定排序依据，reverse决定升序（False）还是降序（True）
lst.copy()	返回列表lst的浅复制

2.2 列表

➢列表的创建

- 使用“=”直接将一个列表赋值给变量即可创建列表对象

```
>>> a_list = ['a', 'b', 'mpilgrim', 'z', 'example']
>>> a_list = []
```

- 也可以使用list()函数将元组、range对象、字符串或其他类型的可迭代对象类型的数据转换为列表。

```
>>> a_list = list((3,5,7,9,11))
>>> a_list
[3, 5, 7, 9, 11]
>>> list(range(1,10,2))
[1, 3, 5, 7, 9]
>>> list('hello world')
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
```

2.2 列表

➢列表的删除

- 当不再使用时，使用**del命令**删除整个列表，如果列表对象所指向的值不再有其他对象指向，**Python**将同时删除该值。

```
>>> a=[1, 2, 3, 4]
>>> del a
>>> a
Traceback (most recent call last):
  File "<pyshell#54>", line 1, in <module>
    a
NameError: name 'a' is not defined
>>>
```

2.2 列表

➢列表元素的增加

（1）可以使用“+”运算符将元素添加到列表中

✓
```
>>> aList = [3,4,5]
>>> aList = aList + [7]
>>> aList
[3, 4, 5, 7]
```

- 严格意义上来讲，这并不是真的为列表添加元素，而是创建了一个新列表，并将原列表中的元素和新元素依次复制到新列表的内存空间。
- 由于涉及大量元素的复制，该操作速度较慢，在涉及大量元素添加时不建议使用该方法。

（2）使用列表对象的append()方法在当前列表尾部追加元素，原地修改列表

- 所谓“原地”，是指不改变列表在内存中的首地址。
- 是真正意义上的在列表尾部添加元素，速度较快。

✓
```
>>> aList.append(9)
>>> aList
[3, 4, 5, 7, 9]
```

2.2 列表

- 对于列表而言，运算符+=和append()有共同点，但又不完全一样
  - ✓
```
>>> x = []
>>> x += '1234'
>>> x
['1', '2', '3', '4']
>>> x += range(3)
>>> x
['1', '2', '3', '4', 0, 1, 2]
```

- Python采用的是基于值的自动内存管理方式，当为对象修改值时，并不是真的直接修改变量的值，而是使变量指向新的值，这对于Python所有类型的变量都是一样的。
  - ✓
```
>>> a = [1,2,3]
>>> id(a)
20230752
>>> a = [1,2]
>>> id(a)
20338208
```

2.2 列表

- 列表中包含的是元素值的引用，而不是直接包含元素值。
  - ✓ 如果是直接修改序列变量的值，则与Python普通变量的情况是一样的
  - ✓ 如果是通过下标来修改序列中元素的值或通过可变序列对象自身提供的方法来增加和删除元素时，序列对象在内存中的起始地址是不变的，仅仅是被改变值的元素地址发生变化，也就是所谓的“原地操作”。

```
>>> a = [1,2,4]
>>> b = [1,2,3]
>>> id(a) == id(b)
False
>>> id(a[0]) == id(b[0])
True
```

```
>>> a = [1,2,3]
>>> id(a)
25289752
>>> a.append(4)
>>> id(a)
25289752
>>> a.remove(3)
>>> a
[1, 2, 4]
>>> id(a)
25289752
>>> a[0] = 5
>>> a
[5, 2, 4]
>>> id(a)
25289752
```

2.2 列表

（3）使用列表对象的extend()方法可以将另一个迭代对象的所有元素添加至该列表对象尾部。

- 通过extend()方法来增加列表元素也不改变其内存首地址，属原地操作。

✓
```
>>> a= [1,2,3]
>>> id(a)
46036232
>>> a.extend([2,3,4])
>>> id(a)
46036232
>>> print(a)
[1, 2, 3, 2, 3, 4]
```

（4）使用列表对象的insert()方法将元素添加至列表的指定位置。

✓
```
>>> a.insert(3, 6)      #在下标为3的位置插入元素6
>>> print(a)
[1, 2, 3, 6, 2, 3, 4]
>>> id(a)
46036232
```

2.2 列表

➢列表元素的增加

- 应尽量从列表尾部进行元素的增加与删除操作。
  - ✓ 列表的insert()可在列表的任意位置插入元素，但由于列表的自动内存管理功能，insert()会引起插入位置之后所有元素的移动，这会影响处理速度。
  - ✓ 类似的还有后面介绍的remove()方法以及使用pop()函数弹出列表非尾部元素和使用del命令删除列表非尾部元素的情况。

2.2 列表

➢列表元素的增加

（5）使用乘法来扩展列表对象，将列表与整数相乘，生成一个新列表，新列表是原列表中元素的重复。

✓
```
>>> aList = [3,5,7]
>>> bList = aList
>>> id(aList)
57091464
>>> id(bList)
57091464
```

```
>>> aList = aList*3
>>> aList
[3, 5, 7, 3, 5, 7, 3, 5, 7]
>>> bList
[3,5,7]
>>> id(aList), id(bList)
(57092680, 57091464)
```

- 当使用*运算符将包含列表的列表重复并创建新列表时，并不是复制子列表值，而是复制已有元素的引用。因此，当修改其中一个值时，相应的引用也会被修改。

✓
```
>>> x = [[None] * 2] * 3
>>> x
[[None, None], [None, None], [None, None]]
>>> x[0][0] = 5
>>> x
[[5, None], [5, None], [5, None]]
```

列表元素的删除

（1）使用del命令删除列表中的指定位置上的元素。

```
>>> aList = [3,5,7,9,11]
>>> del aList[1]
>>> aList
[3, 7, 9, 11]
```

（2）使用列表的pop()方法删除并返回*指定位置*（默认为最后一个）上的元素，如果给定的索引超出了列表的范围则抛出异常。

```
>>> aList = list((3,5,7,9,11))
>>> aList.pop()
11
>>> aList
[3, 5, 7, 9]
>>> aList.pop(5)
IndexError: pop index out of range
```

（3）使用列表对象的remove()方法删除首次出现的指定元素，如果列表中不存在要删除的元素，则抛出异常。

✓
```
>>> aList = [3,5,7,9,7,11]
>>> aList.remove(7)
>>> aList
[3, 5, 9, 7, 11]
```

- 代码编写好后必须要经过反复测试，不能满足于几次测试结果正确

✓
```
>>> x = [1,2,1,2,1,2,1,2,1]
>>> for item in x:
    if item == 1:
        x.remove(item)

>>> x
[2, 2, 2, 2]
```

代码成功地删除了列表中的重复元素，执行结果是完全正确的！

列表元素的删除

```
>>> x = [1,2,1,2,1,1,1]
>>> for item in x:
        if item == 1:
            x.remove(item)

>>> x
[2, 2, 1]
```

- 同样的代码，仅仅是所处理的数据发生了一点变化，然而当循环结束后却发现并没有把所有的“1”都删除，只是删除了一部分。
- 上面这段代码的逻辑是错误的。

- 两组数据的本质区别在于，第一组数据中没有连续的“1”，而第二组数据中存在连续的“1”。出现这个问题的原因是列表的自动内存管理功能。
- 在删除列表元素时，Python会自动对列表内存进行收缩并移动列表元素以保证所有元素之间没有空隙，增加列表元素时也会自动扩展内存并对元素进行移动以保证元素之间没有空隙。每当插入或删除一个元素之后，该元素位置后面所有元素的索引就都改变了。

- 正确的代码

```
>>> x = [1,2,1,2,1,1,1]
>>> for i in range(len(x)-1,-1,-1):                    #从后往前删
    if x[i]==1:
        del x[i]
```

2.2 列表

➢列表元素的访问与计数

- 使用下标直接访问列表元素，如果指定下标不存在，则抛出异常。
  - ✓
```
>>> aList = [1, 2, 3]
>>> aList[4]  #异常
```

- 使用列表对象的**index()**方法获取指定元素首次出现的下标，若列表对象中不存在指定元素，则抛出异常。
  - ✓
```
>>> aList.index(2)
1
>>> aList.index(100) #异常
```

2.2 列表

列表元素的访问与计数

- 使用列表对象的count()方法统计指定元素在列表对象中出现的次数。

```
>>> aList = [3, 4, 5, 5]
>>> aList.count(3)
1
>>> aList.count(5)
2
>>> aList.count(8)
0
```

- 使用in关键字来判断一个值是否存在于列表中，返回结果为“True”或“False”。

```
>>> bList = [[1], [2], [3]]
>>> 3 in bList
False
```

```
>>> aList = [3, 5, 7, 9, 11]
>>> bList = ['a', 'b', 'c', 'd']
>>> (3, 'a') in zip(aList, bList)
True
```

切片操作(slice)

- 切片适用于列表、元组、字符串、range对象等类型，但作用于列表时功能最强大。
- 可以使用切片来截取列表中的任何部分，得到一个新列表，也可以通过切片来修改和删除列表中部分元素，甚至可以通过切片操作为列表对象增加元素。
- 切片使用2个冒号分隔的3个数字来完成：
  - ✓ 第一个数字表示切片开始位置（默认为0）。
  - ✓ 第二个数字表示切片截止（但不包含）位置（默认为列表长度）。
  - ✓ 第三个数字表示切片的步长（默认为1），当步长省略时可以顺便省略最后一个冒号。
- 切片操作不会因为下标越界而抛出异常，而是简单地在列表尾部截断或者返回一个空列表，代码具有更强的健壮性。

2.2 列表

- 列表中的切片操作

函数或方法	描述
ls[i] = x	替换列表ls第i数据项为x
ls[i: j] = lt	用列表lt替换列表ls中第i到j项数据（不含第j项，下同）
ls[i: j: k] = lt	用列表lt替换列表ls中第i到j以k为步的数据
del ls[i: j]	删除列表ls第i到j项数据，等价于ls[i: j]=[]
del ls[i: j: k]	删除列表ls第i到j以k为步的数据

```
>>> a=[1, 2, 3, 4]
>>> a[1:3]=[8, 9]
>>> a
[1, 8, 9, 4]
```

2.2 列表

- 切片操作

```
>>> aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> aList[::]                                    #返回包含所有元素的新列表
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> aList[::-1]                                  #逆序的所有元素
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
>>> aList[::2]                                   #偶数位置，隔一个取一个
[3, 5, 7, 11, 15]
>>> aList[1::2]                                  #奇数位置，隔一个取一个
[4, 6, 9, 13, 17]
>>> aList[3::]                                   #从下标3开始的所有元素
[6, 7, 9, 11, 13, 15, 17]
>>> aList[3:6]                                   #下标在[3, 6)之间的所有元素
[6, 7, 9]
>>> aList[0:100:1]                               #前100个元素，自动截断
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> aList[100:]                                  #下标100之后的所有元素，自动截断
[]
>>> aList[100]                                   #直接使用下标访问会发生越界
IndexError: list index out of range
```

2.2 列表

- 可以使用切片来原地修改列表内容

```
>>> aList = [3, 5, 7]
>>> aList[:2] = [1, 2]                    #替换前2个元素
>>> aList
[1, 2, 7]
>>> aList = list(range(10))
>>> aList
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> aList[::2] = [0]*5                    #替换偶数位置上的元素
>>> aList
[0, 1, 0, 3, 0, 5, 0, 7, 0, 9]
>>> aList[::2] = [0]*3
ValueError: attempt to assign sequence of size 3 to
extended slice of size 5
```

- 使用del与切片结合来删除列表元素

```
>>> aList = [3,5,7,9,11]
>>> del aList[1::2]
>>> aList
[3, 7, 11]                                #删除奇数位置上的元素
```

- 可以使用切片来删除指定位置的元素

```
>>> a=[1,2,3,4,5]
>>> a[1:3]=[]
>>> a
[1, 4, 5]
```

注意：切片赋值时，等号左右两侧的序列长度可以不相等，如果赋值的长度大于切片的长度，会插入新的元素。如果赋值的长度小于切片的长度，会删除一部分元素。

```
>>> a=[1,2,3,4,5]
>>> a[1:3]=[10,20,30,40]
>>> a
[1, 10, 20, 30, 40, 4, 5]
>>> a=[1,2,3,4,5]
>>> a[1:3]=[10]
>>> a
[1, 10, 4, 5]
```

2.2 列表

- 切片返回的是列表元素的浅复制
  - 浅复制指生成一个新的列表，并且把原列表中所有元素的引用都复制到新列表中。
- 如果原列表中只包含整数、实数、复数等基本类型或元组、字符串这样的不可变类型的数据，一般是没有问题的。

```
>>> aList = [3, 5, 7]
>>> bList = aList[::] #切片，浅复制
>>> aList == bList    #两个列表的元素完全一样
True
>>> aList is bList    #但不是同一个对象
False
```

```
>>> id(aList) == id(bList)
False
>>> bList[1] = 8
>>> print(bList, aList)
[3, 8, 7] [3, 5, 7]
```

- 如果原列表中包含列表之类的可变数据类型，由于浅复制时只是把子列表的引用复制到新列表中，因而修改任何一个都会影响另外一个。

```
>>> x = [1, 2, [3,4]]
>>> y = x[:]
>>> x[0] = 5
>>> x
[5, 2, [3, 4]]
>>> y
[1, 2, [3, 4]]
```

```
>>> x[2].append(6)
>>> x
[5, 2, [3, 4, 6]]
>>> y
[1, 2, [3, 4, 6]]
```

- 标准库copy中的deepcopy()函数实现深复制。
- 所谓深复制，是指对原列表中的元素进行递归，把所有的值都复制到新列表中，对嵌套的子列表不再是复制引用。
- 新列表和原列表是互相独立，修改任何一个都不会影响另外一个。

```
>>> import copy
>>> x = [1, 2, [3, 4]]
>>> y = copy.deepcopy(x)
>>> x[2].append(5)
>>> y.append(6)
>>> y
[1, 2, [3, 4], 6]
>>> x
[1, 2, [3, 4, 5]]
```

列表排序

- 使用列表对象的sort()方法进行原地排序，支持多种不同的排序方法

```
>>> aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> import random
>>> random.shuffle(aList)
>>> aList
[3, 4, 15, 11, 9, 17, 13, 6, 7, 5]
>>> aList.sort()                                    #默认是升序排序
>>> aList.sort(reverse = True)                      #降序排序
>>> aList
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
>>> aList.sort(key = lambda x:len(str(x)))  #按转换成字符串的长度排序
>>> aList
[9, 7, 6, 5, 4, 3, 17, 15, 13, 11]
```

2.2 列表

➢列表排序

- 使用内置函数sorted()对列表进行排序并返回新列表

```
>>> aList
[9, 7, 6, 5, 4, 3, 17, 15, 13, 11]
>>> sorted(aList)                                    #升序排序
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> sorted(aList,reverse = True)                     #降序排序
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
```

- 使用列表对象的reverse()方法将元素原地逆序

```
>>> aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> aList.reverse()
>>> aList
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
```

- 使用内置函数reversed()对列表元素进行逆序排列并返回

map、reverse、filter、enumerate、zip访问过的元素不可再次访问！

2.2 列表

- 用于序列操作的常用内置函数

```
Python 3.9.0 Shell
File Edit Shell Debug Options Window Help
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct  5 2020, 15:34:40) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> sum([1,2,3],4)
10
>>> sum([],4)
4
>>> sum([],[1,2,3])
[1, 2, 3]
>>> sum([1,2,3],[])
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    sum([1,2,3],[])
TypeError: can only concatenate list (not "int") to list
>>> sum([[1],[2,3],[4]],[])
[1, 2, 3, 4]
>>>
```

0初始定义：sum([],***)，返回 ***

1.看：求 数值 ∑OR 集合+

2.脱：一层方括号 判断 类型匹配 再 运算

始[])

单选题 1分

设置

此题未设置答案，请点击右侧设置按钮

```
>>>a=sum([[1,2,3,[5]]],[7])
>>>a
```

A [1, 2, 3, 5, 7]

B [7, 1, 2, 3, 5]

C [1, 2, 3, [5], 7]

D [7, 1, 2, 3, [5]]

提交

小测试

探讨！

sum(list(range(3)), 7)

sum(list(range(3)), [7])

sum([list((1,3))],[7])的求值情况！

注意：list(1,3)是错误的写法

用于序列操作的常用内置函数

- len(列表)：返回列表中的元素个数，同样适用于元组、字典、集合、字符串等。
- max(列表)、 min(列表)：返回列表中的最大或最小元素，同样适用于元组、字典、集合、range对象等。
- sum(列表)：对列表的元素进行求和运算，对非数值型列表运算需要指定start参数，同样适用于元组、range。
  - ✓
```
>>> sum(range(1, 11))      #sum()函数的start参数默认为0
55
>>> sum(range(1, 11), 5) #指定start参数为5，等价于5+sum(range(1,11))
60
>>> sum([[1, 2], [3], [4]], [])      #这个操作占用空间较大，慎用
[1, 2, 3, 4]
```

2.2 列表

- zip()函数返回可迭代的zip对象
  - ✓
```
>>> aList = [1, 2, 3]
>>> bList = [4, 5, 6]
>>> cList = zip(a, b)
>>> cList
<zip object at 0x0000000003728908>
>>> list(cList)
[(1, 4), (2, 5), (3, 6)]
```
- enumerate(列表):枚举列表元素，返回枚举对象
  - 其中每个元素为包含下标和值的元组。
  - 该函数对元组、字符串同样有效。
    - ✓
```
>>> for item in enumerate('abcdef'):
        print(item)
(0, 'a')
(1, 'b')
(2, 'c')
(3, 'd')
(4, 'e')
(5, 'f')
```

列表推导式(list comprehensions)

- 列表推导式使用非常简洁的方式来快速生成满足特定需求的列表，代码具有非常强的可读性。
- 列表推导式语法形式为：

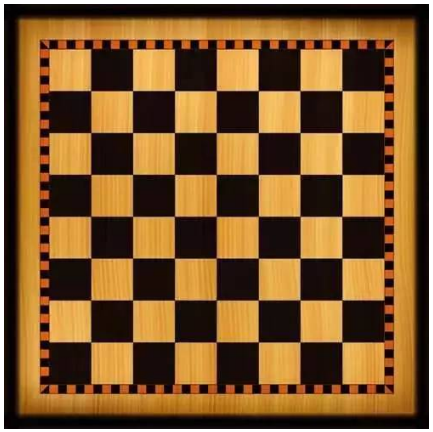
```
[expression for expr1 in sequence1 if condition1
            for expr2 in sequence2 if condition2
            for expr3 in sequence3 if condition3
            ...
            for exprN in sequenceN if conditionN]
```

列表推导式的应用



阿凡提与国王比赛下棋，国王说要是自己输了的话阿凡提想要什么他都可以拿得出来，阿凡提说那就要点米吧。

棋盘一共64个小格子，在第一个格子里放1粒米，第二个格子里放2粒米，第三个格子里放4粒米，第四个格子里放8粒米，以此类推，后面每个格子里的米都是前一个格子里的2倍，一直把64个格子都放满。需要多少粒米呢？



```
>>> sum([2**i for i in range(64)])
18446744073709551615
```

- 列表推导式
  - 列表推导式在内部实际上是一个循环结构，只是形式更加简洁
  - 例如：

```
>>> aList = [x*x for x in range(10)]
```

相当于

```
>>> aList = []
>>> for x in range(10):
    aList.append(x*x)
```

也相当于

```
>>> aList = list(map(lambda x: x*x, range(10)))
```

列表推导式的应用

- 编写算法实现嵌套列表的平铺
  - [[1,2,3], [4,5,6]] → [1, 2, 3, 4, 5, 6]

解答一：

```
>>> vec = [[1, 2, 3], [4, 5, 6]]
>>> result = []
>>> for elem in vec:
    for num in elem:
        result.append(num)

>>> result
[1, 2, 3, 4, 5, 6]
```

解答二：

```
>>> vec = [[1, 2, 3], [4, 5, 6]]
>>> sum(vec, [])
[1, 2, 3, 4, 5, 6]
```

解答三：使用列表推导式实现

```
>>> vec = [[1,2,3], [4,5,6]]
>>> [num for elem in vec for num in elem]
[1, 2, 3, 4, 5, 6]
```

列表推导式的应用

- 列出当前文件夹下所有Python源文件

```
>>> import os
>>> [filename for filename in os.listdir('.')
    if filename.endswith(('.py', '.pyw'))]
```

- 过滤不符合条件的元素

```
>>> aList = [-1,-4,6,7.5,-2.3,9,-11]
>>> [i for i in aList if i>0]
[6, 7.5, 9]
```

- 在列表推导式中使用多个循环，实现多序列元素的任意组合，并且可以结合条件语句过滤特定元素

```
>>> [(x, y) for x in range(3) for y in range(3)]
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]
>>> [(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

2.3 元组

元组(tuple)的定义

- 元组和列表类似，但属于不可变序列，元组一旦创建，用任何方法都不可以修改其元素。
- 元组的定义方式和列表相同，但定义时所有元素是放在一对圆括号“（）”中，而不是方括号中。

元组的创建与删除

- 使用“=”将一个元组赋值给变量

✓

```
>>> aTuple = ('a', 'b', 'mpilgrim', 'z', 'example')
>>> aTuple
('a', 'b', 'mpilgrim', 'z', 'example')
>>> a = (3,)                    #包含一个元素的元组，最后必须多写个逗号
>>> a
(3,)
>>> a = 3,
>>> a
(3,)
>>> x = ()                      #空元组
```

2.3 元组

➢ 元组的创建与删除

- 使用tuple函数将其他序列转换为元组

```
>>> tuple('abcdefg')                    #把字符串转换为元组
('a', 'b', 'c', 'd', 'e', 'f', 'g')
>>> aList
[-1, -4, 6, 7.5, -2.3, 9, -11]
>>> tuple(aList)                        #把列表转换为元组
(-1, -4, 6, 7.5, -2.3, 9, -11)
>>> s = tuple()                         #空元组
>>> s
()
```

- 使用del可以删除元组对象，不能删除元组中的元素

元组与列表的区别

- 元组一旦定义就不允许更改。
- 元组没有append()、extend()和insert()等方法，无法向元组中添加元素。
- 元组没有remove()或pop()方法，也无法对元组元素进行del操作，不能从元组中删除元素。

元组的优点

- 元组的速度比列表更快。如果定义了一系列常量值，而所需做的仅是对它进行遍历，那么一般使用元组而不用列表。
- 元组对不需要改变的数据进行“写保护”将使得代码更加安全。
- 元组可用作字典的“键”，也可以作为集合的元素。列表永远不能当做字典键使用，也不能作为集合的元素，因为列表不是不可变的。



2.3 元组

序列解包(sequence unpacking)

- 可以使用序列解包功能对多个变量同时赋值

```
>>> x, y, z = 1, 2, 3                    #多个变量同时赋值
>>> v_tuple = (False, 3.5, 'exp')
>>> (x, y, z) = v_tuple
>>> x, y, z = v_tuple
>>> x, y, z = range(3)                   #可以对range对象进行序列解包
>>> x, y, z = iter([1, 2, 3])            #使用迭代器对象进行序列解包
>>> x, y, z = map(str, range(3))         #使用可迭代的map对象进行序列解包
>>> a, b = b, a                          #交换两个变量的值
>>> x, y, z = sorted([1, 3, 2])          #sorted()函数返回排序后的列表
>>> a, b, c = 'ABC'                      #字符串也支持序列解包
>>> x = [1, 2, 3, 4, 5, 6]
>>> x[:3] = map(str, range(5))           #切片也支持序列解包
```

序列解包

- 序列解包遍历多个序列

```
>>> keys = ['a', 'b', 'c', 'd']
>>> values = [1, 2, 3, 4]
>>> for k, v in zip(keys, values):
        print((k, v), end=' ')

('a', 1) ('b', 2) ('c', 3) ('d', 4)
```

2.3 元组

使用序列解包遍历enumerate对象

(0, 'a'), (1, 'b'), (2, 'c')

```
>>> x = ['a', 'b', 'c']
>>> for i, v in enumerate(x):
    print('The value on position {0} is {1}'.format(i,v))

The value on position 0 is a
The value on position 1 is b
The value on position 2 is c
```

```
>>> aList = [1,2,3]
>>> bList = [4,5,6]
>>> cList = [7,8,9]
>>> dList = zip(aList, bList, cList)
>>> for index, value in enumerate(dList):
    print(index, ':', value)

0 : (1, 4, 7)
1 : (2, 5, 8)
2 : (3, 6, 9)
```

生成器推导式

- 生成器推导式与列表推导式类似，但是使用圆括号，结果是一个生成器对象。
- 使用生成器对象的元素时，可以根据需要将其转化为列表或元组，也可以使用生成器对象__next__()方法或next()进行遍历。
- 不管用哪种方法访问生成器对象，都无法再次访问已访问过的元素。

2.3 元组

```
>>> g = ((i+2)**2 for i in range(10)) #创建生成器对象
>>> g
<generator object <genexpr> at 0x0000000003095200>
>>> tuple(g)
(4, 9, 16, 25, 36, 49, 64, 81, 100, 121)
>>> list(g)
[]
>>> g = ((i+2)**2 for i in range(10)) #重新创建生成器对象
>>> g.__next__()                      #使用生成器对象的__next__()方法获取元素
4
>>> next(g)                           #使用函数next()获取生成器对象中的元素
16
```

- 使用for循环直接迭代生成器对象中的元素

```
>>> g = ((i+2)**2 for i in range(10))
>>> for item in g:                    #使用循环直接遍历生成器对象中的元素
    print(item, end=' ')

4 9 16 25 36 49 64 81 100 121
```



2.4 字典

- 字典的相关概念
  - 字典是无序、可变序列。
  - 定义字典时，每个元素的键和值用冒号分隔，元素之间用逗号分隔，所有的元素放在一对大括号“{}”中。
  - 字典中的键可以为任意不可变数据，比如整数、实数、复数、字符串、元组等等。
  - **globals()**返回包含当前作用域内所有全局变量和值的字典
  - **locals()**返回包含当前作用域内所有局部变量和值的字典

2.4 字典

字典类型的操作

函数和方法	描述
<d>.keys()	返回所有的键信息
<d>.values()	返回所有的值信息
<d>.items()	返回所有的键值对
<d>.get(<key>,<default>)	键存在则返回相应值，否则返回默认值
<d>.pop(<key>,<default>)	键存在则返回相应值，同时删除键值对，否则返回默认值
<d>.popitem()	随机从字典中取出一个键值对，以元组(key, value)形式返回
<d>.clear()	删除所有的键值对
del <d>[<key>]	删除字典中某一个键值对
<key> in <d>	如果键在字典中返回True，否则返回False



2.4 字典

字典的创建与删除

- 使用=将一个字典赋值给一个变量
  - ✓
```
>>> a_dict = {'server': 'db.diveintopython3.org', 'database': 'mysql'}
>>> a_dict
{'database': 'mysql', 'server': 'db.diveintopython3.org'}
>>> x = {}                                    #空字典
>>> x
{}
```

- 使用dict()函数利用已有数据创建字典
  - ✓
```
>>> keys = ['a', 'b', 'c', 'd']
>>> values = [1, 2, 3, 4]
>>> dictionary = dict(zip(keys, values))
>>> dictionary
{'a': 1, 'c': 3, 'b': 2, 'd': 4}
>>> x = dict()                                #空字典
>>> x
{}
```



2.4 字典

字典的创建与删除

- 使用dict()函数根据给定的键、值创建字典

```
>>> d = dict(name='Dong', age=37)
>>> d
{'age': 37, 'name': 'Dong'}
```

- 以给定内容为键，创建值为空的字典

```
>>> adict = dict.fromkeys(['name', 'age'])
>>> adict
{'age': None, 'name': None}
```

- 可以使用del删除整个字典



2.4 字典

字典元素的读取

- 以键作为下标可以读取字典元素，若键不存在则抛出异常

```
>>> aDict = {'name':'Dong', 'age':37}
>>> aDict['name']
'Dong'
>>> aDict['tel']                          #键不存在，抛出异常
```



2.4 字典

- 使用字典对象的get方法获取指定键对应的值，并且可以在键不存在的时候返回指定值。

```
>>> print(aDict.get('address'))
None
>>> print(aDict.get('address', 'SDIBT'))
SDIBT
```

- 使用字典对象的items()方法可以返回字典的键、值对
- 使用字典对象的keys()方法可以返回字典的键
- 使用字典对象的values()方法可以返回字典的值



2.4 字典

```
>>> aDict={'name':‘Dong’, ‘age':37}
>>> for item in aDict.items():      #输出字典中所有元素
        print(item)

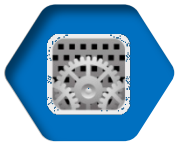
('age', 37)
('name', ‘Dong')

>>> for key in aDict:                #不加特殊说明，默认输出键
    print(key)

age
name

>>> for key, value in aDict.items():        #序列解包用法
    print(key, value)

age 37
name Dong
```



2.4 字典

字典元素的添加与修改

- 当以指定键为下标为字典赋值时：1）若键存在，则可以修改该键的值；2）若不存在，则表示添加一个键、值对。

```
>>> aDict['age'] = 37                    #修改元素值
>>> aDict
{'age': 37, 'name': 'Li'}
>>> aDict['address'] = 'hefei'           #增加新元素
>>> aDict
{'age': 37, 'address': 'hefei', 'name': 'Li'}
```

- 使用字典对象的**update()**方法将另一个字典的键、值对添加到当前字典对象。

```
>>> aDict={'age': 37, 'score': [98, 97], 'name': 'Li'}
>>> aDict.update({'a':'a','b':'b'})
>>> aDict
{'age': 37, 'score': [98, 97], 'name': 'Li', 'a': 'a', 'b': 'b'}
```

字典元素的添加与修改

- 使用del删除字典中指定键的元素
- 使用字典对象的clear()方法来删除字典中所有元素
- 使用字典对象的pop()方法删除并返回指定键的元素
- 使用字典对象的popitem()方法删除并返回字典中的一个元素



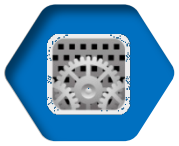
2.4 字典

字典应用案例

- 编程：生成包含1000个随机字符的字符串，统计每个字符的出现次数。

```
import string
import random
x = '0123456789abcdefg'
# y = [random.choice(x) for i in range(1000)]
y = random.choices(x, k=1000)
z = ''.join(y)
d = dict()                          #使用字典保存每个字符出现次数
for ch in z:
    d[ch] = d.get(ch, 0) + 1
print(d)
```

```
{'5': 64, 'd': 56, 'a': 71, '0': 67, '4': 62, 'b': 73, 'c': 49, '6': 54, '7': 60
, '1': 55, 'f': 66, '8': 57, '3': 54, 'g': 66, '9': 47, 'e': 52, '2': 47}
>>>
```



2.4 字典

- 也可以使用collections模块的defaultdict类来实现。

```
>>> from collections import defaultdict
>>> frequences = defaultdict(int)
>>> frequences
defaultdict(<type 'int'>, {})
>>> for item in z:
    frequences[item] += 1
>>> frequences.items()
```

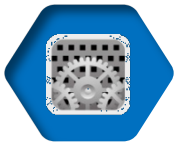
字符串中的字母第一次出现时，字典中没有该字母，调用int()为其提供一个默认值0，加法操作将计算出每个字母出现的次数。

- 使用collections模块的Counter类可以快速实现这个功能，并且提供更多功能，例如查找出现次数最多的元素。

Counter类的目的是用来跟踪值出现的次数。它是一个无序的容器类型，以字典的键值对形式存储，其中元素作为key，其计数作为value。

```
>>> from collections import Counter
>>> frequences = Counter(z)
>>> frequences.items()
>>> frequences.most_common(1)
[('A', 22)]
```

#出现次数最多的一个字符

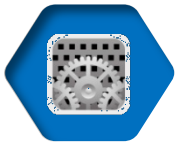


2.4 字典

字典应用案例

- Python内置字典是无序的，如果需要一个可以记住元素插入顺序的字典，可以使用collections.OrderedDict。

```
>>> x = dict()                          #无序字典
>>> x['a'] = 3
>>> x['b'] = 5
>>> x['c'] = 8
>>> x
{'b': 5, 'c': 8, 'a': 3}
>>> import collections
>>> x = collections.OrderedDict()       #有序字典
>>> x['a'] = 3
>>> x['b'] = 5
>>> x['c'] = 8
>>> x
OrderedDict([('a', 3), ('b', 5), ('c', 8)])
```



2.4 字典

2.3.6 字典推导式(dict comprehensions)

```
>>> s = {x:x.strip() for x in ('  he  ', 'she    ', '    I')}
>>> s
{'  he  ': 'he', '    I': 'I', 'she    ': 'she'}
>>> for k, v in s.items():
    print(k, ':', v)

  he   : he
    I : I
she     : she
>>> {i:str(i) for i in range(1, 5)}
{1: '1', 2: '2', 3: '3', 4: '4'}
>>> x = ['A', 'B', 'C', 'D']
>>> y = ['a', 'b', 'b', 'd']
>>> {i:j for i,j in zip(x,y)}
{'A': 'a', 'C': 'b', 'B': 'b', 'D': 'd'}
```



2.5 集合

集合的基本概念

- 集合是无序、可变序列，使用一对大括号界定，元素不可重复，同一个集合中每个元素都是唯一的。
- 集合中只能包含数字、字符串、元组等不可变类型（或者说可哈希）的数据，而不能包含列表、字典、集合等可变类型的数据。

集合的创建与删除

- 直接将集合赋值给变量

```
>>> a = {3, 5}
>>> a.add(7)                    #向集合中添加元素
>>> a
{3, 5, 7}
```

集合的创建与删除

- 使用set将其他类型数据转换为集合

```
>>> a_set = set(range(8,14))
>>> a_set
{8, 9, 10, 11, 12, 13}
>>> b_set = set([0, 1, 2, 3, 0, 1, 2, 3, 7, 8])    #自动去除重复
>>> b_set
{0, 1, 2, 3, 7, 8}
>>> c_set = set()                                   #空集合
>>> c_set
set()
```

- 当不再使用某个集合时，删除方法包括：
  - 可以使用del命令删除整个集合；
  - 集合对象的pop()方法弹出并删除其中一个元素；
  - remove()方法直接删除指定元素；
  - clear()方法清空集合。

2.5 集合

- 集合类型有10个操作函数或方法

函数或方法	描述
S.add(x)	如果数据项x不在集合S中，将x增加到s
S.clear()	移除S中所有数据项
S.copy()	返回集合S的一个拷贝
S.pop()	随机返回集合S中的一个元素，如果S为空，产生KeyError异常
S.discard(x)	如果x在集合S中，移除该元素；如果x不在，不报错
S.remove(x)	如果x在集合S中，移除该元素；不在产生KeyError异常
S.isdisjoint(T)	如果集合S与T没有相同元素，返回True
len(S)	返回集合S元素个数
x in S	如果x是S的元素，返回True，否则返回False
x not in S	如果x不是S的元素，返回True，否则返回False



2.5 集合

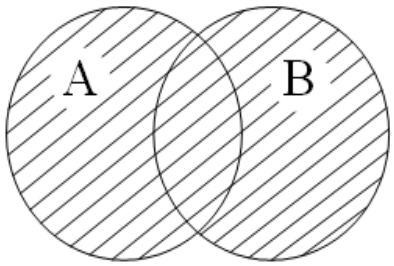
DMiC -HFUT-

- 集合类型有10个操作符

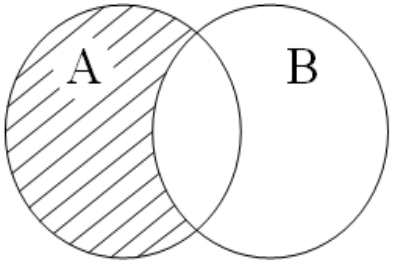
操作符	描述
S – T 或 S.difference(T)	返回一个新集合，包括在集合S中但不在集合T中的元素
S-=T或S.difference_update(T)	更新集合S，包括在集合S中但不在集合T中的元素
S & T或S.intersection(T)	返回一个新集合，包括同时在集合S和T中的元素
S&=T或S.intersection_update(T)	更新集合S，包括同时在集合S和T中的元素。
S^T或s.symmetric_difference(T)	返回一个新集合，包括集合S和T中元素，但不包括同时在其中的元素
S=^T或 s.symmetric_difference_update(T)	更新集合S，包括集合S和T中元素，但不包括同时在其中的元素
S|T或S.union(T)	返回一个新集合，包括集合S和T中所有元素
S=|T或S.update(T)	更新集合S，包括集合S和T中所有元素
S<=T或S.issubset(T)	如果S与T相同或S是T的子集，返回True，否则返回False，可以用S<T判断S是否是T的真子集
S>=T或S.issuperset(T)	如果S与T相同或S是T的超集，返回True，否则返回False，可以用S>T判断S是否是T的真超集

2.5 集合

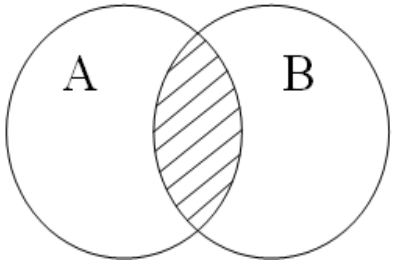
- 上述操作符表达了集合类型的4种基本操作，交集（&）、并集（|）、差集（-）、补集（^）
- 操作逻辑与数学定义相同



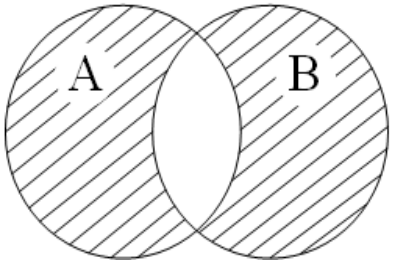
A|B



A-B



A&B



A^B

集合运用案例

✓ 例：生成不重复随机数的效率比较。

```
import random
import time

def RandomNumbersByList(number, start, end):
    data = []
    n = 0
    while True:
        element = random.randint(start, end)
        if element not in data:
            data.append(element)
            n += 1
        if n == number:
            break
    return data
```

```
def RandomNumbersBySet(number, start, end):
    data = set()
    while True:
        data.add(random.randint(start, end))
        if len(data) == number:
            break
    return data
```

```
begin, end = 1, 100000
num = 50000 # 要获取的不重复数字个数
rep = 10    # 重复测试次数
for ran in (RandomNumbersByList, RandomNumbersBySet):
    start = time.time()
    for i in range(rep):
        ran(num, begin, end)
    print(ran.__name__, time.time()-start)
```

RandomNumbersByList 247.10713362693787
RandomNumbersBySet 1.2580716609954834



2.5 集合

- 集合推导式

```
>>> s = {x.strip() for x in ('  he  ', 'she    ', '    I')}
>>> s
{'I', 'she', 'he'}
```

- 再谈内置方法sorted()
  - 列表对象提供了sort()方法支持原地排序，而内置函数sorted()返回新列表，并不对原列表进行任何修改。
  - sorted()方法可以对列表、元组、字典、range对象等进行排序。
  - 列表的sort()方法和内置函数sorted()都支持key参数实现复杂排序要求。



2.5 集合

内置方法sorted()

```
>>> persons = [{'name':'Li', 'age':40}, {'name':'Li', 'age':37},
               {'name':'Dong', 'age':43}]
>>> print(persons)
[{'name': 'Li', 'age': 40}, {'name': 'Li', 'age': 37}, {'name': 'Dong',
'age': 43}]
#使用key来指定排序依据，先按姓名升序排序，姓名相同的按年龄降序排序
>>> print(sorted(persons, key=lambda x:(x['name'], -x['age'])))
[{'name': 'Dong', 'age': 43}, {'name': 'Li', 'age': 40}, {'name': 'Li',
'age': 37}]
>>> phonebook = {'Linda':'7750', 'Bob':'9345', 'Carol':'5834'}
>>> from operator import itemgetter
>>> sorted(phonebook.items(), key=itemgetter(1))
[('Carol', '5834'), ('Linda', '7750'), ('Bob', '9345' )]
```

itemgetter函数用于获取对象的哪些维的数据

- 内置方法sorted()
  - ✓ 问题描述：有一个整数列表，要求调整元素顺序，把所有奇数都放到前面，偶数都放到后面。

```
>>> from random import randint
>>> x = [randint(1,100) for i in range(20)]
>>> x
[19, 32, 76, 82, 23, 63, 38, 50, 20, 30, 39, 14, 19, 50, 81, 27, 77, 12, 55, 29]
>>> sorted(x, key=lambda item:item%2==0)
[19, 23, 63, 39, 19, 81, 27, 77, 55, 29, 32, 76, 82, 38, 50, 20, 30, 14, 50, 12]
```

上文回顾



- 数据类型 (Data Type)
  - 数字类型(Number Type)
    - 整型 (int)
    - 浮点类型 (float)
    - 复数类型 (complex)
  - Python序列(Python Sequence)
    - 序列解包 (sequence unpacking)
    - 有序序列 (Ordered Sequence)
      - 列表(list)
        - 切片(slice)
          - 列表推导式(list comprehensions)
      - 元组 (tuple)
        - 生成器推导式(generator comprehensions)
      - 字符串 (str)
      - zip、map等对象
    - 无序序列 (Unordered Sequence)
      - 字典(dict)
        - 字典推导式(dict comprehensions)
      - 集合 (set)
        - 集合推导式(set comprehensions)

堆 (Heap)

堆是非线性的树形的数据结构，包括：最大堆与最小堆。

heapq库中的堆默认是最小堆。

```
>>> import heapq                          #heapq和random是Python标准库
>>> import random
>>> data = list(range(10))
>>> data
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> random.shuffle(data)                  #随机打乱顺序
>>> data
[6, 1, 3, 4, 9, 0, 5, 2, 8, 7]
>>> heap=[]
>>> for n in data:                        #建堆
    heapq.heappush(heap,n)

>>> heap
[0, 2, 1, 4, 7, 3, 5, 6, 8, 9]
>>> heapq.heappop(heap)                   #出堆，自动重建
0
```



2.6 复杂数据结构

堆

```
>>> myheap = [1,2,3,5,7,8,9,4,10,333]
>>> heapq.heapify(myheap)                #建堆
>>> myheap
[1, 2, 3, 4, 7, 8, 9, 5, 10, 333]
>>> heapq.heapreplace(myheap,6)          #弹出最小元素，同时插入新元素
1
>>> myheap
[2, 4, 3, 5, 7, 8, 9, 6, 10, 333]
>>> heapq.nlargest(3, myheap)            #返回前3个最大的元素
[333, 10, 9]
>>> heapq.nsmallest(3, myheap)           #返回前3个最小的元素
[2, 3, 4]
```



2.6 复杂数据结构

队列(Queue)

- 使用列表来模拟队列结构

```
>>> x = [1, 2, 3, 4]
>>> x.pop(0)
1
>>> x.pop(0)
2
>>> x.append(5)
>>> x.pop(0)
3
>>> x.pop(0)
4
>>> x.pop(0)
5
>>> x.pop(0)
Traceback (most recent call last):
  File "<pyshell#8>", line 1, in <module>
    x.pop(0)
IndexError: pop from empty list
```

- queue是Python标准库

```
>>> import queue
>>> q = queue.Queue()
>>> q.put(0)                #入队
>>> q.put(1)
>>> q.put(2)
>>> q.queue
deque([0, 1, 2])
>>> q.get()                 #出队
0
>>> q.queue
deque([1, 2])
>>> q.get()
1
>>> q.queue
deque([2])
```

队列(Queue)

1)“后进后出”队列LILO (Last In Last Out)

```
>>> from queue import Queue      #LILO队列
>>> q = Queue()                  #创建队列对象
>>> q.put(0)                     #在队列尾部插入元素
>>> q.put(1)
>>> print(q.queue)               #显示队列中所有元素
deque([0, 1])
>>> q.get()                      #返回并删除队列头部元素
0
```

2)“后进先出”队列LIFO (Last In First Out)

```
>>> from queue import LifoQueue #LIFO队列
>>> q = LifoQueue()              #创建LIFO队列对象
>>> q.put(2)                     #在队列尾部插入元素
>>> q.put(3)
>>> print(q.queue)
[2, 3]
>>> q.get()                      #返回并删除队列尾部元素
3
>>> q.get()
2
>>> q.get()                      #对空队列调用get()方法会阻塞当前线程
```



2.6 复杂数据结构

3)优先队列(Priority Queue)

优先队列具有最高级先出的行为特征。通常采用堆数据结构来实现。

```
>>> from queue import PriorityQueue #优先级队列
>>> q = PriorityQueue()             #创建优先级队列对象
>>> q.put(3)                        #插入元素
>>> q.put(8)                        #插入元素
>>> q.put(100)
>>> q.queue                         #查看优先级队列中所有元素
[3, 8, 100]
>>> q.put(1)                        #插入元素，自动调整优先级队列
>>> q.put(2)
>>> q.queue
[1, 2, 100, 8, 3]
>>> q.get()                         #返回并删除优先级最低的元素
1
>>> q.get()                         #请多执行几次该语句并观察返回的数据
2
>>> q.get()
3
>>> q.get()
8
>>> q.get()
100
```

2.6 复杂数据结构

优先队列存入数据可以设置优先级，设置的优先级的值越小，则表明优先级越高，越先被取出。

```
import queue
q = queue.PriorityQueue()
q.put([1, 10])   # 1是级别最高的
q.put([40, 20])
q.put([3, 'Python'])
q.put([5, 30])
if __name__ == '__main__':
    while not q.empty():  # 不为空时候执行
        print("get data =", q.get())
```

```
get data = [1, 10]
get data = [3, 'Python']
get data = [5, 30]
get data = [40, 20]
>>>
```

双端队列(Double-ended Queue)

- Python标准库collections提供了双端队列deque

```
>>> from collections import deque
>>> q = deque(maxlen=5)                    #创建双端队列
>>> for item in [3, 5, 7, 9, 11]:          #添加元素
    q.append(item)
>>> q.append(13)                           #队列满，自动溢出
>>> q.append(15)
>>> q
deque([7, 9, 11, 13, 15], maxlen=5)
>>> q.appendleft(5)                        #从左侧添加元素，右侧自动溢出
>>> q
deque([5, 7, 9, 11, 13], maxlen=5)
```

栈(Stack)

- 栈是一种“后进先出（LIFO）”或“先进后出（FILO）”的数据结构。
- Python列表本身就可以实现栈结构的基本操作。例如，列表对象的append()方法是在列表尾部追加元素，类似于入栈操作；pop()方法默认是弹出并返回列表的最后一个元素，类似于出栈操作。
- 但是直接使用Python列表对象模拟栈操作并不是很方便，例如当列表为空时再执行pop()出栈操作时则会抛出一个不很友好的异常；另外，也无法限制栈的大小。

栈(Stack)

- 可以直接使用列表来实现栈结构

```
>>> myStack = []
>>> myStack.append(3)
>>> myStack.append(5)
>>> myStack.append(7)
>>> myStack
[3, 5, 7]
>>> myStack.pop()
7
>>> myStack.pop()
5
>>> myStack.pop()
3
>>> myStack.pop()
出错
```

封装列表实现栈结构

```
class Stack:
    def __init__(self, size = 10):
        self._content = []                  #使用列表存放栈的元素
        self._size = size                   #初始栈大小
        self._current = 0                   #栈中元素个数初始化为0

    def empty(self):
        self._content = []
        self._current = 0

    def isEmpty(self):
        if not self._content:
            return True
        else:
            return False
```

- 封装列表实现栈结构

```
def setSize(self, size):
    #如果缩小栈空间，则删除指定大小之后的已有元素
    if size < self._current:
        for i in range(size, self._current)[::-1]:
            del self._content[i]
        self._current = size
    self._size = size

def isFull(self):
    if self._current == self._size:
        return True
    else:
        return False
```

- 封装列表实现栈结构

```
def push(self, v):
    if len(self._content) < self._size:
        self._content.append(v)
        self._current = self._current+1  #栈中元素个数加1
    else:
        print('Stack Full!')

def pop(self):
    if self._content:
        self._current = self._current-1  #栈中元素个数减1
        return self._content.pop()
    else:
        print('Stack is empty!')
```



2.6 复杂数据结构

封装列表实现栈结构

```
def show(self):
    print(self._content)

def showRemainderSpace(self):
    print('Stack can still PUSH ',
          self._size-self._current,
          ' elements.')

if __name__ == '__main__':
    print('Please use me as a module.')
```

自定义栈的用法

```
>>> import Stack
>>> x = Stack.Stack()
>>> x.push(1)
>>> x.push(2)
>>> x.show()
[1, 2]
>>> x.pop()
2
>>> x.show()
[1]
>>> x.showRemainderSpace()
Stack can still PUSH  9  elements.
>>> x.isEmpty()
False
>>> x.isFull()
False
```

链表(Linked List)

- 可直接使用列表来实现

```
>>> linkTable = []
>>> linkTable.append(3)
>>> linkTable.append(5)
>>> linkTable
[3, 5]
>>> linkTable.insert(1,4)
>>> linkTable
[3, 4, 5]
>>> linkTable.remove(linkTable[1])
>>> linkTable
[3, 5]
```

二叉树(Binary Tree)

- 使用代码中的类BinaryTree创建的对象不仅支持二叉树的创建以及前序遍历、中序遍历与后序遍历等三种常用的二叉树节点遍历方式，还支持二叉树中任意“子树”的遍历。

```
class BinaryTree:
    def __init__(self, value):
        self.__left = None
        self.__right =  None
        self.__data = value

    def insertLeftChild(self, value):  #创建左子树
        if self.__left:
            print('__left child tree already exists.')
        else:
            self.__left = BinaryTree(value)
            return self.__left
```

```
def preOrder(self):                          #前序遍历
    print(self.__data)                       #输出根节点的值
    if self.__left:
        self.__left.preOrder()               #遍历左子树
    if self.__right:
        self.__right.preOrder()              #遍历右子树

def postOrder(self):                         #后序遍历
    if self.__left:
        self.__left.postOrder()
    if self.__right:
        self.__right.postOrder()
    print(self.__data)

def insertRightChild(self, value): #创建右子树
    if self.__right:
        print('Right child tree already exists.')
    else:
        self.__right = BinaryTree(value)
        return self.__right
```

```
def show(self):
        print(self.__data)


 def inOrder(self):                    #中序遍历
     if self.__left:
         self.__left.inOrder()
     print(self.__data)
     if self.__right:
         self.__right.inOrder()

  if __name__ == '__main__':
      print('Please use me as a module.')
```

- 自定义二叉树用法

```
>>> import BinaryTree
>>> root = BinaryTree.BinaryTree('root')
>>> b = root.insertRightChild('B')
>>> a = root.insertLeftChild('A')
>>> c = a.insertLeftChild('C')
>>> d = c.insertRightChild('D')
>>> e = b.insertRightChild('E')
>>> f = e.insertLeftChild('F')
>>> root.inOrder()
C  D  A  root  B  F  E
>>> root.postOrder()
D  C  A  F  E  B  root
>>> b.inOrder()
B  F  E
```

有向图(Directed Graph)

```
def searchPath(graph, start, end):
    results = []
    __generatePath(graph, [start], end, results)
    results.sort(key = lambda x:len(x))
    return results

def __generatePath(graph, path, end, results):
    current = path[-1]
    if current == end:
        results.append(path)
    else:
        for n in graph[current]:
            if n not in path:
                #path.append(n)
                __generatePath(graph, path + [n], end, results)
```

有向图(Directed Graph)

```
def showPath(results):
    print('The path from ',results[0][0], ' to ',
          results[0][-1], ' is:')
    for path in results:
        print(path)

if __name__ == '__main__':
    graph = {'A':['B', 'C', 'D'],
                  'B':['E'],
                  'C':['D', 'F'],
                  'D':['B', 'E', 'G'],
                  'E':['D'],
                  'F':['D', 'G'],
                  'G':['E']}
    r1 = searchPath(graph, 'A', 'D')
    showPath(r1)
    r2 = searchPath(graph, 'A', 'E')
    showPath(r2)
```

1.6 本章小结

内容回顾

- 可变序列：列表、字典、集合
- 不可变序列: 元组、字符串、zip等对象
- 复杂数据结构：堆、队列、栈、链表、二叉树、图



思考

- 如何编写复杂数据结构的Python代码？